

MLOps and AI Systems Engineering

From notebook to production: the practices, tools, and pipelines that keep a machine learning model reliable.



COURSE OVERVIEW

Everything lives in one place



GitHub repository

github.com/aligunesgit/MlOps

Every module's lecture notes, exercises, and source code



Course site (GitHub Pages)

aligunesgit.github.io/MlOps

The same material, nicely rendered, searchable, with diagrams



Atlas-OIS

Announcements, deadlines, and the course platform

COURSE OVERVIEW

The roadmap: 12 weeks

Week	Module	Topic
1	M1	Development Environment
2	M2	Organisation and Version Control
3	M3	Reproducibility
4	M4	Debugging, Profiling and Logging
5	M5	Continuous Integration
6		Project Sprint A
7	M6	The Cloud
8	M7	Deployment
9	M8	Monitoring
10	M9	Scalable Applications
11	M10	Extra
12		Project Sprint B (final)

COURSE OVERVIEW

What this course covers

Foundations (M1-M3)

Command line, uv, VS Code, PyTorch; Git, DVC, Cookiecutter, good coding practice; Docker and Hydra config files

Automate and scale (M4, M5, M9)

Debugging, profiling, W&B logging; pytest, GitHub Actions, pre-commit, CML; distributed data loading, DDP, scalable inference

Ship it to the cloud (M6-M8)

Google Cloud setup, compute and storage; FastAPI, ONNX, BentoML, a Streamlit frontend; Evidently drift detection, Prometheus and Cloud Monitoring

Beyond the core (M10 + the project)

Documentation with MkDocs, Optuna, high performance clusters; and one group project that runs the whole semester, Sprint A through Sprint B

Project ideas, if you want a starting point

Text

- Tweet classification (DTU)
- English to German translation (DTU)
- Scientific paper classification with GNNs (DTU)
- Spam / phishing message detection
- Product or movie review sentiment
- Duplicate question / FAQ matching

Image

- Rice type classification (DTU)
- Plant / leaf disease classification
- Safety equipment detection (helmet, vest)
- Chest X-ray or skin lesion classification

Audio

- Keyword spotting (Speech Commands)
- Music genre classification from spectrograms

Tabular & time series

- Energy consumption or sensor forecasting
- IoT anomaly detection

Pick any problem you want. Start small: under 1GB of data, a simple model, then add complexity. The goal is demonstrating MLOps practice, not chasing state of the art.

Learning objectives



Be comfortable navigating and scripting from the command line



Understand what a package manager solves, and set up a project with uv



Configure an editor (VS Code) for Python development, including AI-assisted completion



Install PyTorch correctly and implement a basic training/evaluation loop

§1 · WHY THIS MODULE EXISTS

Everything else in this course assumes you already have this.

The command line, a clean Python environment, a real editor, and a working PyTorch install are not MLOps specifically. They are the floor everything else in this course stands on.

Anatomy of a command

```
[prompt] $ command --option argument
```

PROMPT

Current directory and a shell symbol, often with environment info

COMMAND

The executable instruction itself, for example **ls** or **cd**

OPTIONS / FLAGS

Modifiers starting with - or -- that change behavior

ARGUMENTS

The inputs the command actually operates on

Commands you will use constantly

Command	What it does
<code>cd / pwd</code>	Change directory / print the current one
<code>ls -l</code>	List a folder's contents in detail
<code>which</code>	Locate where a program actually lives
<code>cat</code>	Print a file's contents
<code>less</code>	Page through a file's contents
<code>top</code>	Live system resource usage
<code>wget</code>	Download a file from a URL
<code>></code>	Redirect a command's output into a file

Scripts and environment variables

Bash scripting

```
#!/bin/bash
for i in {1..10}; do
    python script.py
done
```

A for-loop turns ten manual runs into one reusable file.

Environment variables

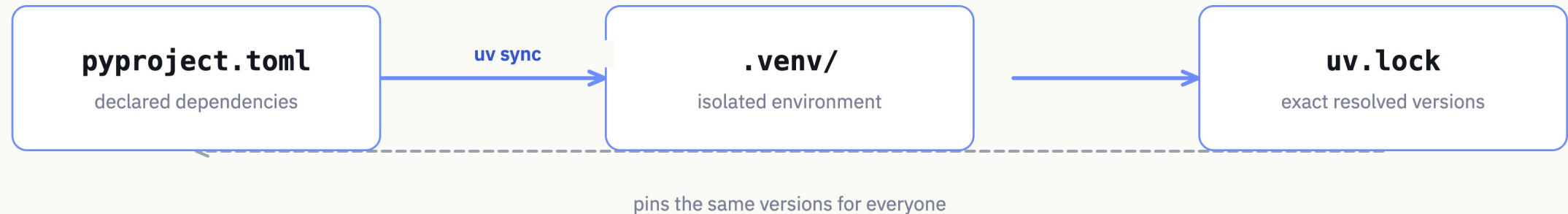
```
export MY_VAR=hello
echo $MY_VAR
```

```
import os
os.environ["MY_VAR"]
```

A .env file plus python-dotenv is the durable version of the same idea.

Why not just pip install everything?

Two projects that need different versions of the same library cannot both be satisfied by one global install: the last one installed silently wins. A virtual environment gives every project its own isolated package directory. This course uses **uv**, the fastest of the group.



The uv commands you will actually use

Command	What it does
<code>uv init</code>	Scaffolds a new project (pyproject.toml, .venv/, README.md)
<code>uv add <package></code>	Adds a runtime dependency
<code>uv add --dev <package></code>	Adds a development-only dependency (tests, linters)
<code>uv sync</code>	Installs exactly what pyproject.toml / uv.lock specify
<code>uv run <script></code>	Runs a script inside the project's environment
<code>uv python pin 3.13</code>	Pins the Python version for this project
<code>uv export --format requirements.txt</code>	Exports a plain requirements.txt for tools that still need one

A notebook is not a project

This course uses **VS Code**. Four extensions cover the Python workflow:

Python

Pylance

Jupyter

Python Environments

Converting a prototype notebook to a script matters the moment a model needs to leave it:

```
jupyter nbconvert --to=script notebook.ipynb
```

AI-assisted completion

GitHub Copilot suggests code inline as you type, and Ctrl+I opens an inline chat scoped to selected code.

The responsibility stays with you. A plausible-looking suggestion is not the same as a correct one.

PyTorch, and getting the install right

TensorFlow

JAX

PyTorch

Most published research and competition-winning models are built on PyTorch. Installation size is worth getting right up front:

6.6 GB

`pip install torch`, naive GPU build



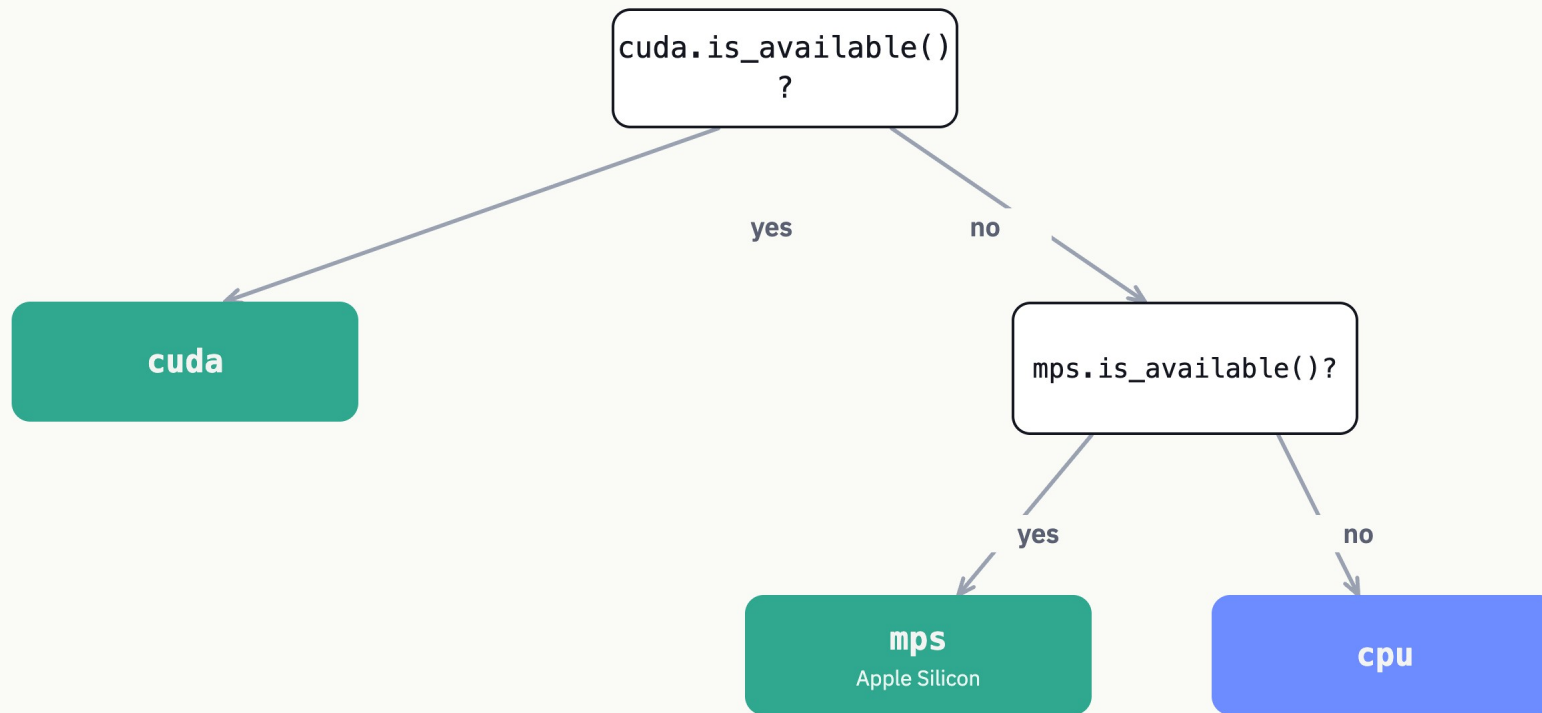
700 MB

CPU-only, pointed at the right index

```
uv add torch --index https://download.pytorch.org/whl/cpu
```

Never hardcode the device

The same code needs to run on a laptop with no GPU at all.



Three lines every training loop needs

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Line	What it does	If you forget it
zero_grad()	Clears gradients from the previous step	Gradients accumulate and explode
backward()	Computes gradients for the current batch	No gradients exist to update
step()	Applies the computed gradients to the weights	The model never learns

REQUIRED
PROJECT

Corrupted MNIST baseline

The project that ties §2 through §5 together into one working pipeline.

`model.py`

A small CNN: Conv2d layers, pooling, dropout, a final linear layer to 10 classes

`data.py`

Loads the .pt files, adds the channel dimension a CNN expects, returns train/test datasets

`main.py`

A typer CLI with train and evaluate commands

85%+ target test accuracy

```
uv run main.py train --lr 1e-4
uv run main.py evaluate model.pt
```

MODULE 1 · WRAP-UP

Before Module 2 starts, you should have

- ✓ A terminal you can navigate and script without hesitation
- ✓ A uv-managed project with a locked, reproducible environment
- ✓ VS Code configured for Python, with AI completion as a tool, not a crutch
- ✓ A working PyTorch training loop that hits 85%+ on corrupted MNIST

Module 2 restructures that project into a proper, version-controlled package.